Week 7 Home Work

Submission Deadline: 5 June 2026, 11:59 PM

---

1. School Exam Score Tracker

Concepts: 3D Lists · While Loop · Indexing · Data Types

Problem Statement:

- A school tracks exam scores for 2 grades, each with 3 subjects, and 4 students per subject.
- Store all scores in a 3D list: [grade][subject][student].
- Grade names: ['Grade 10', 'Grade 11']
- Subject names: ['Math', 'Science', 'English']
- Use a while loop with index to print scores for each grade and subject.
- For each subject, calculate and print the average score (rounded to 1 decimal).
- Find and print the highest score overall, and which grade and subject it belongs to.
- Count students who scored below 50 in any subject (failing students).

Acceptance Criteria:

- Define a 3D list with shape [2][3][4].
- Use three levels of index access: scores[g][s][st].
- Calculate averages using a while loop — no built-in sum() or avg().
- Track highest score and its location using variables.
- Count failing students using a counter variable inside nested while loops.

Expected Output:

```
======================================
EXAM SCORE REPORT
======================================
Grade: Grade 10
Math | Scores: 72 85 61 90 | Average: 77.0
Science | Scores: 55 48 79 83 | Average: 66.3
English | Scores: 88 91 74 65 | Average: 79.5

Grade: Grade 11
Math | Scores: 95 88 77 69 | Average: 82.3
Science | Scores: 60 72 85 91 | Average: 77.0
English | Scores: 45 78 83 90 | Average: 74.0

Highest score: 95
Grade: Grade 11 | Subject: Math | Student 1

Failing students (below 50): 2
```

2. Hotel Room Booking System

Concepts: 2D Lists · While Loop · String Methods · Conditionals

Problem Statement:

- A hotel manages 5 rooms, each with 3 fields: [room_type, price_per_night, is_available].
- Pre-fill the 2D list with data for 5 rooms.
- Room types: Standard, Deluxe, Suite, Family, Executive.
- Use a while loop to display all rooms and their availability status.

- Allow the user to book a room by entering a room number (1-5); enter 0 to exit.
- If the room is already booked (is_available == 0), print 'Room not available'.
- If available, set is_available to 0 and print booking confirmation with total cost for 3 nights.
- After exiting, print total rooms booked and total revenue collected.

Acceptance Criteria:

- Define a 2D list with 5 rows, each as [room_type, price, is_available].
- Use index access rooms[i][0], rooms[i][1], rooms[i][2].
- Use a while loop with index to print the room table.
- Use while True with break when input is 0.
- Update rooms[i][2] = 0 after a successful booking.
- Track bookings count and revenue with variables outside the loop.

Expected Output:

```
======================================
HOTEL ROOM AVAILABILITY
======================================
No. | Type | Price/Night | Status
--------------------------------------
1 | Standard | 5000 | Available
2 | Deluxe | 8000 | Available
3 | Suite | 15000 | Available
4 | Family | 10000 | Available
5 | Executive | 20000 | Available

Enter room number to book (0 to exit): 2
Booking confirmed! Deluxe room for 3 nights.
Total cost: 24000 yen

Enter room number to book (0 to exit): 2
Room not available.

Enter room number to book (0 to exit): 0

Total rooms booked : 1
Total revenue : 24000 yen
```

3. Weather Data Tracker

Concepts: 3D Lists · While Loop · Indexing · Data Types

Problem Statement:

- A weather agency tracks data for 3 cities, across 4 seasons, recording 3 values each: [temperature, humidity, rainfall].
- Store all data in a 3D list: [city][season][measurement].
- City names: ['Tokyo', 'Osaka', 'Kyoto']
- Season names: ['Spring', 'Summer', 'Autumn', 'Winter']
- Use a while loop to print the full weather report for each city and season.
- Find the hottest season (highest temperature) for each city.
- Find the city with the highest total annual rainfall.

Acceptance Criteria:

- Define a 3D list with shape [3][4][3].
- Use index access: weather[c][s][0] for temp, [1] for humidity, [2] for rainfall.

- Use nested while loops with index for all traversals — no sum() or max().
- Track hottest season per city using a variable reset each city iteration.
- Track total rainfall per city using an accumulator variable.

Expected Output:

```
========================================
WEATHER REPORT
========================================
City: Tokyo
Spring | Temp: 18C Humidity: 60% Rain: 120mm
Summer | Temp: 34C Humidity: 80% Rain: 180mm
Autumn | Temp: 22C Humidity: 65% Rain: 140mm
Winter | Temp: 8C Humidity: 50% Rain: 60mm
Hottest season: Summer (34C)

City: Osaka
Spring | Temp: 20C Humidity: 62% Rain: 110mm
Summer | Temp: 36C Humidity: 82% Rain: 170mm
Autumn | Temp: 24C Humidity: 64% Rain: 130mm
Winter | Temp: 9C Humidity: 52% Rain: 55mm
Hottest season: Summer (36C)

City: Kyoto
Spring | Temp: 19C Humidity: 61% Rain: 105mm
Summer | Temp: 35C Humidity: 79% Rain: 165mm
Autumn | Temp: 23C Humidity: 63% Rain: 125mm
Winter | Temp: 7C Humidity: 51% Rain: 50mm
Hottest season: Summer (35C)

City with highest annual rainfall: Tokyo (500mm)
```

4. Warehouse Stock Manager

Concepts: 2D Lists · While Loop · String Methods · Lists · Conditionals

Problem Statement:

- A warehouse manager adds product stock records using a while loop.
- Each record contains: [product_name, category, quantity, unit_price].
- Accept records until the manager types 'done'.
- Store each record as a list inside a 2D list.
- Use .title() on product name and .upper() on category before storing.
- After entry, print the full inventory table using a while loop with index.
- For each item, compute total stock value (quantity x unit_price).
- Classify stock: quantity >= 100 --> 'High', quantity >= 50 --> 'Medium', below 50 --> 'Low'.
- Print the product with the highest stock value at the end.

Acceptance Criteria:

- Use while True with break when name == 'done'.
- Store records as: inventory.append([name, category, qty, price]).
- Access using inventory[i][0] through inventory[i][3].
- Use if/elif/else for stock level classification.
- Track highest value product using a variable and index.

Expected Output:

```
Enter product name (or 'done'): bolt
Enter category: hardware
Enter quantity: 250
Enter unit price: 12.5
Enter product name (or 'done'): cable
Enter category: electronics
Enter quantity: 40
Enter unit price: 350.0
Enter product name (or 'done'): done
============================================================
WAREHOUSE INVENTORY REPORT
============================================================
Product | Category | Qty | Unit Price | Total Value | Stock
------------------------------------------------------------
Bolt | HARDWARE | 250 | 12.5 | 3125.0 | High
Cable | ELECTRONICS | 40 | 350.0 | 14000.0 | Low
------------------------------------------------------------
Highest value product: Cable (14000.0 yen)
```

5. University Course Registration System

Concepts: 3D Lists · 2D Lists · While Loop · String Methods · Conditionals · Data Types

Problem Statement:

- A university has 3 departments, each offering 4 courses.
- Each course stores: [course_name, credits, max_seats, enrolled].
- Store this in a 3D list: [department][course][field].
- Department names: ['Engineering', 'Business', 'Arts']
- Print the full course catalog for all departments using nested while loops with index.
- Mark each course as 'Open' if enrolled < max_seats, else 'Full'.
- Use a while loop to allow a student to register: enter department number (0-2) and course
- number (0-3). Enter -1 to exit.
- If the course is full, print 'Registration failed: Course is full'.
- If open, increment enrolled by 1 and confirm registration.
- After exiting, print total successful registrations made during the session.

Acceptance Criteria:

- Define a 3D list with shape [3][4][4].
- Use index access courses[d][c][3] to read/update enrolled count.
- Use nested while loops with index to print the catalog.
- Use while True with break when input == -1.
- Update courses[d][c][3] += 1 after a successful registration.
- Count and print total successful registrations after the loop.

Expected Output:

```
==========================================
UNIVERSITY COURSE CATALOG
==========================================
Department: Engineering
0. Algorithms | Credits: 3 | Seats: 30 | Enrolled: 28 | Open
```

```
1. Networks | Credits: 3 | Seats: 25 | Enrolled: 25 | Full
2. Databases | Credits: 2 | Seats: 35 | Enrolled: 10 | Open
3. AI Basics | Credits: 4 | Seats: 20 | Enrolled: 20 | Full

Department: Business
0. Marketing | Credits: 3 | Seats: 40 | Enrolled: 38 | Open
1. Finance | Credits: 3 | Seats: 30 | Enrolled: 30 | Full
2. Management | Credits: 2 | Seats: 35 | Enrolled: 20 | Open
3. Economics | Credits: 4 | Seats: 25 | Enrolled: 25 | Full

Department: Arts
0. History | Credits: 3 | Seats: 30 | Enrolled: 15 | Open
1. Philosophy | Credits: 3 | Seats: 20 | Enrolled: 20 | Full
2. Literature | Credits: 2 | Seats: 25 | Enrolled: 10 | Open
3. Fine Arts | Credits: 4 | Seats: 20 | Enrolled: 18 | Open

Enter department number (0-2, or -1 to exit): 0
Enter course number (0-3): 1
Registration failed: Course is full.

Enter department number (0-2, or -1 to exit): 0
Enter course number (0-3): 0
Registered successfully for Algorithms! (Enrolled: 29/30)

Enter department number (0-2, or -1 to exit): -1

Total successful registrations this session: 1
```